

Lec 17 Practical Exercise - 1

Lec 17 Practical Exercise - 1

Lecture	17 / 63 + intro
Duration	35:59
Captions	English (auto-generated)
Words	5,439
Video	5C95atwLrAM
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:01] Hello all. Welcome to the hands-on series on fundamentals of generative AI and large language models, theory and practice. In today's session, we'll be implementing the different types of encoders, namely the shallow or the basic autoencoder. We have the deep autoencoder, sparse autoencoder, and denoising autoencoder. So, let us see the implementation of the varieties of autoencoders. So, initially importing the necessary libraries. So, we have different libraries to be imported through which we we'll be able to access different functions. So, firstly we import numerical Python, that is NumPy as NP, which is required for processing of arrays or dealing with numbers. Then, we import the matplotlib.pyplot

[00:50] as PLT, which is required for data visualization purpose. Later, we have we have to import the TensorFlow as TF, which is required for dealing with neural networks. It is a deep learning framework. Then, from tensorflow.keras and the dataset, we import the MNIST dataset, which is basically a handwritten digit dataset, which has numbers varying from 0 to 9. So, it is 10 different classes dataset. And also, we import the dense layer and the input layer. So, dense layer is basically the output layer or a flattened layer, which is required at the output side. And to define the input dataset size, we need this input layer.

[01:35] And later, we also import the model, which is required for building or creating the different neural networks. So, once the importing of necessary libraries is being done, we load the dataset. So, as mentioned earlier, we are dealing with the MNIST dataset. So, we are trying to load this MNIST dataset into two different variables, namely the training set and the test set. So, we define two variables X train and the X test. This underscore is been mentioned here because we are ignoring the labels.

[02:06] We are not doing any classification here. Autoencoder is required in order to reconstruct the original image. So, we just need uh different images or samples of images and nothing to do with their labels. Therefore, we ignore their labels. So, in order to ignore the labels, we have to just mention the underscore here. And in the training set whatever we consider X train that has to be converted into floating data type and then we are normalizing it so that we have values in a particular range, maybe in the range of zero to one or maybe in the range of minus one to one. So, when we scale the data or normalize the data, it is easier for the model to learn the patterns or learn the data. So, in order

[02:49] to make the learning process easier we are normalizing the data. So, same thing to do with the test data as well. So, test data is converted into floating data type and then we normalize. So, in order to normalize, we divide by 255. So, we have uh we have two different 255 different levels because it is the image data. And then we have to reshape the data. So, obtained data after normalization, we will reshape based on the length of the training as well as the test data set into 784 different pixels. So, how do we get the 784 pixels is the original data set what we have, MNIST data set it has 60,000 training samples 10,000 test samples and each of the samples has 28 cross 28 pixels. So, we represent each

[03:42] of the sample or the image having 28 rows and 28 columns. So, when we multiply this 28 into 28, we get this representation of 784. So, basically we are converting the 2D matrix 28 cross 28 into 784 vector. So, that we can extract the 784 different features, input features. So, once this reshaping of the data set is done, we print the X train as well as we print the X test. So, we are able to observe the output here.

[04:12] We have 60,000 samples in the training data set and having 784 different features. Same way, in the test data set, we have 10,000 samples and 784 different features. So, now we have this in the vector form. And this vector form is required because we are making use of the flattened layer or the dense layer or the output layer, we should be having a vector format. That's why we have to reshape the data. So, once this pre-processing is done, we are going into the different types of autoencoders. So, starting with the shallow autoencoder, which is also called as a basic autoencoder.

[04:51] So, as we have seen in the theory sessions, autoencoder comprises of two

parts, the encoder and the decoder part. So, when we have only one encoder and one decoder, it is the shallow or the basic autoencoder. And based on the different types based on the different values of encoders and decoders we have, we have different types of autoencoders. So, first coming to the shallow shallow autoencoder, we need to define the input shape.

[05:18] So, input shape, which is defined using the input function. So, shape equal to 784. We have already considered the 784 different input features. So, we are saving in a variable called as input_image. Now, as mentioned earlier, we have one encoder and one decoder. Encoder, at the output side, we have the dense layer, which is defined by 64 64 different neurons. So, what we are trying to do is compress the data. Initially, we compress the data and decompress the data. So, that will help us in the reconstruction of the original image or get back the original image.

[05:55] So, 784 different uh features, which are represented using 784 neurons, are now compressed into 64 neurons. So, this will help us in saving the memory as well, uh the storage requirement. So, 64 neurons, and then uh we make use of the ReLU activation functions. Different kinds of activation functions can be used. So, based on the necessity or based on the advantages of various uh activation functions, we have made use of ReLU activation function on the input image. And at the decoding side, we have once again have the dense layer, which is the output layer. And at the decoding side, we want to reconstruct or we want to get back the original image of 784 neurons or 784 features. So, therefore, we mention 784

[06:41] neurons here. And the activation function used is sigmoid. So, here sigmoid is sufficient because we uh need to define only two values, that is zero and one, because we have already normalized the given input in the range zero zero to one. So, sigmoid activation function is uh very much uh possible in this case. So, this we uh we implement this dense layer on the encoded variable. So, encoding and the decoding part is done.

[07:11] So, we have initially compressed the data and now decompress the data. So, let us see how to represent the reconstructed data. So, for that, I'm building a model, nothing but an auto encoder uh model here on the input image as well as the decoded representation of the input image. So, now we define or we have defined the model. Then, we compile the data. So, compilation of the data will be done, and we make use of the Adam optimizer, again based on its advantages over several other optimizers.

[07:44] And we have the loss function, which is binary cross entropy. Since we have already normalized the data in the range 0 to 1, so we basically call it as two classes. So, binary cross entropy would be our loss function to use in this case. So, once we compile this model, we will take a summary of this model. What exactly which are the layers being used? What exactly the computation or the operations that has been taken place here.

[08:12] So, the Keras model on the functional API, we can see it here in the summary. Now, the different layers what we have defined as initially the input layer followed by first dense layer, which takes care of the encoding part. And the second dense layer, which takes care of the decoding part. Now, let us see the output shape of each of the layers. So, in the input layer at the output shape, we have none {comma} 784. So, 784 different input features we have taken, so that's 784 here.

[08:43] And number of computations at the input layer will be equal to zero because we are not going to update any weights at the input layer. So, the operations being here or the parameters being used will be equal to zero. Now, at the first dense layer, output shape is none {comma} 64. And number of parameters or trainable parameters will be equal to 50,240. And at the second dense layer, we have the output shape as once again 784 because we have reconstructed the image.

[09:14] And the number of parameters will be equal to 50,960. So, now let us see how to get these trainable parameters or what are the operations that take place under dense layer one and dense layer two. » So, number of parameters Number of parameters will be based on the input that we have into the dense layer neurons plus the bias. So, we have seen the input shape size as 784 into the number of neurons will be equal to 64 and the bias associated with will be equal to 64. So, this is going to give the figure as 50,000 240.

[10:36] Same thing to do with This is dense layer one. Now, when we are talking about dense layer two the number of parameters will be based on what input that is going to the dense layer two into the neurons plus the bias associated with it. So, we have used a 64 neurons at dense layer one. 64 into At the dense layer two, we have the number of neurons as 784 plus the bias associated with it is 784. So, this will give us 50,960.

[11:31] So, these are number of trainable parameters. So, this is how the trainable parameters will be calculated. So, when we add up these two parameters, we get 1 lakh 1,200 different trainable parameters. So, once we have built this shallow autoencoder, let us get fit this on the training data and then see what is the prediction that has been done or how the reconstruction takes place here. So, fit function applied on the training data set. So, we mention training data set twice because one is required at the encoding part, other one required at the decoding part.

[12:13] So, we mention it twice. Unlike when we build other neural networks, we write only once this training data set. But, in this case, encoder as well as decoder both requires the input. Therefore, we measure we mention X train as well as X train. So, epochs is again dependent on the user. User is free to use the number of epochs. So, we have kept the number of epochs as five because it is easy for training. And the number of computation will also be less in this case.

[12:46] So, epochs refers to the number of complete pass through the complete train-

ing data set. So, we have five different passes over the complete data set. And then we have the batch size as 256. So, batch size refers to the number of training samples that we considered. We need not take the complete training data set. Instead, what we can do is take different number of samples at different batches and update the weights at different batches. So, what happens is updation of weights takes place at every batch. So, this will be very easy even in the back propagation.

[13:24] So, we consider the the of batches as 256 over 60,000 training samples. And let us shuffle the number of training samples that we have because a shuffling will always generalize the model. In case we if we take the samples in the serial order, what happens is the model will learn it in a series or it will it will understand the pattern in a serial manner manner. Instead, we have a random kind of shuffling, then what happens is it will not try to memorize the data. It will not try to memorize the data set.

[13:55] So, this will be very helpful in reducing the overfitting condition or it will help in generalizing the model. So, now our validation data set is nothing but the test data set. So, once again, we mention this X to test X X to test it twice because we need one on the encoder side and other one on the decoder side. So, the result of epochs, five different epochs is here. We try to measure the validation loss. So, at the first epoch, we see that the validation loss is 0.16 and then coming to epoch number two, it has reduced to 0.12. Epoch three, it is 0.1. Epoch four, it is 0.09. And coming to epoch five, it has further reduced to 0.09.

[14:39] So, at different epochs, we can see the loss being reduced. Now, coming to plotting of the original image and the reconstructed image reconstructed image that is got by obtaining that is got by taking the auto encoder that is the shallow auto encoder. So, shallow auto encoder is doing some prediction on the X test. Let us save that in a variable called as decoded images. So, let me try to you know, plot 10 different figures. So, I define n equal to 10.

[15:14] The figure function, under the figure function, we have fixed size argument. 20 refers to the width and four refers to the height of the images. So, for I in the range N, so for I different images in the range 10, when we write this, we will be able to access 10 different images, random images. So, the image index will be 0 to 9. So, such such images, 10 different images, let me have two different plots for it. In the first subplot, let me have the original image. In the second subplot, let me have the reconstructed image.

[15:51] So, subplot two refers to two different rows and refers to 10 different columns because we have to represent 10 different images here. So, that can be seen here. Two different rows and 10 different columns being mentioned. And at every iteration, I plus one, we are moving from zero to image to the first image, second image. Likewise, we are covering 10 different images in the range zero to nine here. So, we want

to show the X test starting from the first iteration varying from zero to nine. And let us reshape the image as 28 cross 28 pixels. Unlike the first case what we had, 784 vector representation, we are back converting it into 28 cross 28. Let the color map be gray color.

[16:41] And I'm turning off this axis because we need not name anything in the X or the Y axis. Therefore, we are trying to turn off the axis here. Same thing to do with the decoded image, that is a reconstructed image. So, once again, subplot, two rows and 10 different columns, I varying from the first image up to the last image. So, I plus one plus N. So, I'm trying to show the decoded images, all varying from zero to nine. Once again, we will also reshape the reconstructed image, that is 28 cross 28, and the color map, let it be equal to gray.

[17:17] Once again, turning off the X axis and the Y axis because we need not name anything. We need not give any label to the X and the Y axis. And uh showing that image. So, first row refers to the original image that was present in the MNIST data set. And second row refers to the reconstructed image. So, we can see that with some loss. Uh so, we have seen we have run the uh a data set for five different epochs with certain loss. It is able to reconstruct the image. So, we can see little amount of blurring in each of the cases. But, there is almost a perfect reconstruction of the original image.

[17:56] So, autoencoder in our case has helped in reconstruct them reconstructing the original image. Now, coming to the second type of autoencoder, which is the deep autoencoder. As the name suggest, the network is little deep. That That is why the name is deep autoencoder because the number of encoders and decoders we have here is more compared to the shallow network. So, in the shallow autoencoder, we had only one encoder and one decoder.

[18:26] Whereas, in case of deep autoencoder, we can have multiple encoders and multiple decoders. So, what we have done here is taking the input image, uh which is defined using the input function. Once again, the number of features will be 784. So, we have three encoder part and three decoder part here. So, for every encoding, there should be a decoder. So, we define the encoder and the decoder once again using the dense layer. So, whenever we talk about dense layer, we should be having a vector format of data. That's why we have 256 neurons, further compressed to 128 neurons, further compressed to 64 neurons. So, compression takes place here at the encoding part and at the decoding part once again decompression

[19:18] takes place. So, we start with 128 neurons which moves on to 256 neurons and then we get back the original image with 784 different neurons. The activation functions at different layers are ReLU. Only at the last decoding part we consider the sigmoid activation function because as mentioned earlier we will normalize the data zero and one so to have only two ranges to have a minimum value and the maximum

value sigmoid activation function is much more sufficient.

[19:52] So, this is happening in the deep autoencoder. So, we build the model on the input image and on the decoded uh layers. We compile the model based on the optimizer which is the Adam optimizer and the loss function that we have defined is the binary cross-entropy and then we have the summary for this deep autoencoder. So, we can see the model summary here different layers first being the input layer and then we have the different dense layers and the last dense layer will give you give us the reconstructed image.

[20:28] So, in the first layer there is no computation happening because the because it is just the input layer it is just used to define the shape of the input image. So, number of parameters will be equal to zero because the number of computations are zero. And layer by layer the number of parameters will increase. So, we have already seen how to calculate the parameters. The formula remains the same. Number of parameters will be equal to whatever input that has been defined at that particular layer into the neurons which has been defined at that layer plus the bias associated with those neurons. So, when we try to to at different layers we get the values the same parameter values. That is 2 lakh 9 2 lakh 960 parameters,

[21:14] 32,896 parameters, 8,256 uh parameters followed by 8,320, then 33,000 uh 24 different parameters. And at the last layer, we have 2 lakh 1,488 parameters. So, when we sum up, the number of trainable parameters are this huge. That is 4 lakh 84,944 different parameters. So, these many computations or these many uh weight calculations or weight applications will take place. Now, let us see uh the epoch or the training. So, once again, for the encoder and the decoder, we need two different training data set. X train, once again, the same X train. Let us define the number of epochs as five, the batch size as 256, and the validation data is nothing but the test data set.

[22:15] test data set. So, uh now we have the various uh loss values validation loss values. 0.13 further reduced to 0.1, then we have 0.1029, 0.00968, 0.0937. So, it is further reduced. Now, coming to the prediction part, nothing but the reconstruction. The same way what we have done for the shallow autoencoder, we follow the same procedure wherein we take two different rows, 10 different columns, because we are going to reconstruct or show the representation for 10 different reconstructed images. So, the code remains the same, and we will be able to see the reconstruction. So, when we compare the loss values, we can say that the in the case of shallow autoencoder, the loss value was 0.09.

[23:07] In case of deep autoencoder, the loss value is 0.093. So, there's a slight difference between the loss values. The since this is a deep architecture, we are not able to get uh the proper loss values or the reduced loss values. So, we can see the reconstructed images. Now coming to the third type of autoencoder, which is the

sparse autoencoder. The name sparse is because of the sparsity function uh or the sparsity feature being included in the sparse autoencoder. So, sparse encoder has a regularization or L1 regularization by default. Unlike our deep autoencoder or the shallow autoencoder does not make use of any regularization technique by default, but the sparse encoder has a regularization technique by default,

[24:03] which is the L1 regularization. Now, the regularization values could be anything. It could be 10 to the power of -1, -2, -3. It can vary based on how much we want to regularize the model or how much we want to generalize the model. So, in our case, we have used uh the regularization value of 10 to the power of -4. So, based on a trial and error basis or based on the user uh the regularization values could be different.

[24:30] So, let us uh build this sparse autoencoder. So, from Keras uh then Keras and TensorFlow, we have this importing of the regularizers, different regularizers required. So, we define the input image uh in the input layer. The shape is 784. Then, we have one encoder and one decoder. At the encoding part, we have the dense layer with 64 different neurons. Activation functions will be ReLU. Uh we are trying to define this regularizer regularizer here. So, regularizer is the L1 regularization technique being used of the order 10 to the power of minus four, which is uh made to be enough uh fit on the input image.

[25:13] Then at the decoding part, we have the dense layer with 784 neurons. Activation functions will be uh sigmoid, which is already told earlier the importance of sigmoid activation function, which is applied on the encoded variable. So, sparse autoencoder model development based on the input as well as the decoded layer. Once we compile, we make use of the optimizer, which is the Adam optimizer, and the loss function is binary cross entropy. And then we take to we get the summary of this model. So, we had one input layer followed by two dense layers. So, number of parameters at the input layer will be equal to zero. At the first dense layer, it is 50,000 to 40. At the second dense layer, it is 50,960.

[26:01] So, once again, the parameters are calculated based on the input into the neurons at that particular layer plus the bias associated. So, in this in this model, the number of trainable parameters are 101,200. So, we fit this sparse autoencoder on the training data, number of epochs being five. Batch size, we have used the same thing earlier, 256. And validation data will be X_test as well as once again X_test, both for the encoder and the decoder.

[26:35] So, let us see the validation loss here, 0.6, 0.5, 0.5, 0.46, and 0.43. It is slightly uh uh increased value when compared to the basic autoencoder or the deep autoencoder. The reason here is we have tried to generalize the model. So, what happens in case of generalizing or regularizing the model is, the model will not learn each and every sample. We try to cut short. We try to remove some of the neurons.

So, when we remove some of the neurons, it is nothing but now some of the input features are being removed.

[27:12] So, we try to work only with the 64 different 64 different neurons here. So, what happens in case of L1 regularization is on the loss function, we introduce this regularization parameter, which is λ into the absolute value of the different weights. So, these are the weights associated with different batches. We have 234 batches based on 256 batch size. So, every weight has to be multiplied with this regularization parameter. So, when we try to multiply this, there will be some sparsity, which is introduced.

[28:06] That means all the 784 neurons are not involved in the training. Instead, we try to bring down the neurons into 64. That means we remove some of the neurons. So, which is adding sparsity in the model. So, when we try to remove some of the neurons, what happens is during the training, some of the data memorization will be lost. That's why it is giving more loss to us. Now, coming to the visualization part. So, to images and we try to predict this sparse auto encoder on the X test data. So, we make use of the figure function. Once again, the number of iterations or number of images that we take are 10. Subplot function, two rows and 10 columns.

[29:01] Number of iterations are based on the number of images that we are trying to display. So, we see that most of the reconstruction has failed because the loss is maximum in this case. So, this is not so good technique or not so good auto when compared to the other two auto encoders. Now, coming to the fourth type of auto encoder, which is the denoising auto encoder. So, what we try to do in this type of auto encoder is once again, we try to generalize the model by adding some noise content into the original image. So, we have already seen that we have MNIST data set with the 60,000 training samples and 10,000 test samples. Now, these test samples are original. To the original data set, we try to add some noise. So, in our case,

[29:51] what we have done is we have added some normal distributed noise or it can be also called as the Gaussian noise. So, when we have this Gaussian noise, we try to maintain a mean of zero and standard deviation of one. So, based on the noise factor and the normally distributed data or the normal noise, we can call it as a Gaussian noise, we are trying to modify the training data set. So, in order to do this, first we initialize the noise factor as 0.5. The noise factor can vary from the range zero to one. If we are taking zero, we are adding very minimal noise or the effect of that noise on the data set will be very minimal. If we make this noise factor as one, we are going to add very heavy noise to the

[30:37] original data set. So, in our case, what we have done is we have added a moderate noise. That's why the noise factor will be equal to 0.5. So, now noise factor will be equal to 0.5. Now, I define a new variable called as `X_train_noisy`, which will be equal to the original data set, that is `X_train`, plus the noise factor, which is 0.5, into

the normally distributed data, which is obtained from our random function, which is available in the NumPy package.

[31:08] So, `np.random.normal`, here location will be equal to zero. So, location means we take the mean value as zero. So, we assume that the most of the data is centered around the zero value, that is mean value. So, mean will be equal to zero, nothing but location will be equal to zero. And the scale equal to one means the standard deviation. For a Gaussian noise, the standard deviation will be equal to one. Therefore, the scale in this case will be equal to one.

[31:37] And then the size of the data set is nothing but the shape that is involved in the training data set. So, we are not going to change the size. It is completely dependent on the original data set. Same thing to do with the test data set. We make use of a variable called as `X_test_noisy`, which is dependent on the original test data, plus the noise factor into the Gaussian noise, which is introduced in the test data set as well.

[32:05] So, now what we try to do is the noisy data which we have taken in both in the training and the test data set, we try to clip it. So, clip from the value zero to one. That means if any value is less than zero, any value is less than zero, we try to clip it at zero. Any value greater than zero, we try to clip it at zero. And any value in the range zero to one will remain the same. Any value lying between zero to one will remain the same.

[32:56] So that we don't have too much of negative values as well as too much of positive values. So we bring down to a proper scale. So this procedure is called as clipping of the noise or clipping of the data. So once we have defined our training as well as test data based on the noise factor, then we define the input shape and we make use of two layers of encoding and two layers of decoding. So dense layer with 128 neurons, dense layer with 64 neurons. Uh once again at the decoding part, we have 128 followed by seven uh 84. Relu activation function and at the end it is the sigmoid activation function. So we build the model on the input data and on the decoded data, we compile it based on the optimizer Adam

[33:39] and the loss function being binary cross entropy. So when this when we fit this on the training data, noisy training data, as well as uh test data set which is noisy and uh run five different epochs, we come across this kind of uh validation loss. So 0.18, 0.15, 0.14, so on up to 0.13. So if we had more number of epochs, maybe the validation loss would have been very minimal. But one more thing to note here is this is having a regularization. This is a generalized model because we are going to add some noise. So it is not that the model is learning only the original data set. So we have seen here we make use of `X_train_noisy` as well as `X_train`. Same thing. `X_test_noisy` as well as `X_test`.

[34:28] So we are not training too much only on the original data set but we also

add some noise content to it. So, it's a very generalized model. So, when we have generalized model, we may have to compromise on the loss. So, finally, prediction done or reconstruction which has to be done on the noisy data set. So, the plotting function remains the same, two rows and 10 columns. When we plot it, we see that on the noisy original data set, we have got back the reconstructed image which is blurred only, but the noise content is been removed. So, perfect reconstruction has been taken place even though the our original data set had some noise content.

[35:12] So, this is all about the types of auto encoders what we have discussed. So, shallow auto encoder which has only one encoder and one decoder. The second one was deep auto encoder which had a several or multiple encoding and the decoding parts. Then, we discussed about the sparse auto encoder which had the L1 regularization technique by default. And the fourth one which was the denoising auto encoder wherein the noise factor was been added with some Gaussian noise as well.

[35:49] So, this is all about the types of auto encoder. Thank you.